

Guidelines for CAU Computer Science Graduation Projects: An IMRAD-IEEE Implementation Template

Student Name 1
Central Asian University
Engineering School
Department of Computer Science
Email: student1@centralasian.uz

Student Name 2
CCentral Asian University
Engineering School
Department of Computer Science
Email: student2@centralasian.uz

Supervisor Name
Position, Central Asian University
Engineering School
Department of Computer Science
Email: supervisor@centralasian.uz

Abstract—The abstract should be a single paragraph (150-250 words) that serves as a standalone summary. It must state the problem being solved, the specific technical approach or "stack" used, the core results of your testing (e.g., "The system achieved 95% accuracy" or "reduced latency by 20%"), and a concluding sentence on the project's impact. Avoid citations in this section.

Index Terms—Computer Science, Software Development, IEEE Style, IMRAD, Technical Documentation.

I. INTRODUCTION

The Modern software engineering practices emphasize the importance of modularity and scalability [1]. Recent trends in cloud computing have shifted towards microservices to handle high-traffic demands [2].

A. Related Work

Existing research in cloud-based architectures has largely focused on monolithic designs. For instance, Smith et al. [3] implemented a centralized encryption module, but it suffered from high latency. Similarly, the framework proposed in [2] utilizes microservices but lacks the automated load balancing required for real-time mobile applications. Our project builds upon these concepts by introducing a decentralized hashing mechanism that addresses the latency issues found in previous implementations.

B. Problem Statement

Clearly define the "pain point." Are current systems too slow? Is there a lack of automation in a specific industry? For CS students, the problem is often a technical inefficiency, a security vulnerability, or the non-existence of a specific digital tool for a target user group.

C. Objectives

List what you intend to achieve. The primary objective is usually to design and develop the system. Specific objectives should be "SMART" (Specific, Measurable, Achievable, Relevant, and Time-bound), such as "To implement a real-time encryption module using AES-256" or "To evaluate system performance under a load of 1,000 concurrent users."

II. METHODS

This section explains how you built the solution. In Computer Science, this is the most critical part for reproducibility. You must describe the Software Development Life Cycle (SDLC) followed, whether it was Agile, Scrum, or Waterfall.

A. System Architecture

The system follows a modular three-tier architecture as shown in Fig. 1. By separating the concerns into Presentation, Logic, and Data layers, the system achieves better scalability and easier debugging.

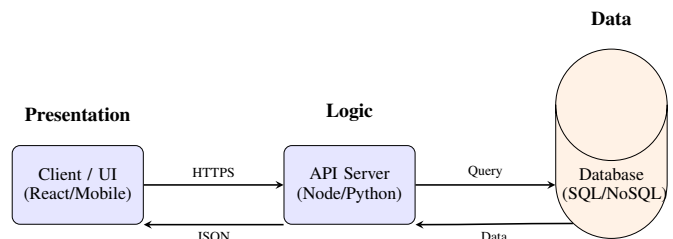


Fig. 1. System Architecture and Data Flow

B. Requirements Analysis

Specify the Functional Requirements (what the system does, e.g., "The system shall allow users to reset passwords") and Non-Functional Requirements (performance, security, and usability, e.g., "The system shall respond within 2 seconds").

C. Implementation Schedule

The timeline ensures the project was managed systematically. The Gantt chart as shown in Fig. 2 below illustrates the allocation of time across the project phases.

D. Technology and Instruments

The user interface was developed using the React framework due to its component-based architecture [4]. To ensure data integrity, encryption was implemented based on established security protocols [3].

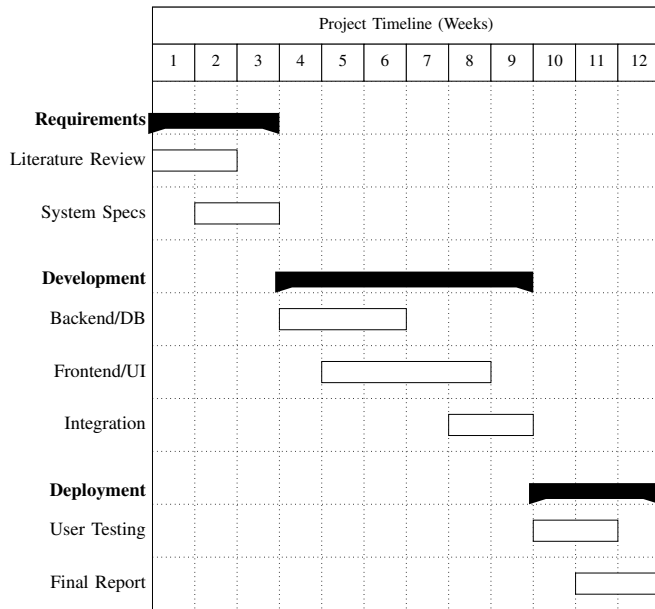


Fig. 2. Project Development Schedule (SDLC)

III. RESULTS

The Results section presents the raw evidence that your system works. Avoid interpreting the data here; simply present it clearly.

A. Implementation Outputs

This subsection should contain high-quality screenshots of your primary user interfaces. If the project is algorithmic, provide snippets of the core logic or pseudocode. Explain what each figure demonstrates about the system's functionality.

B. Testing and Evaluation

Present your testing results. For CS projects, this typically includes:

- **Unit Testing:** Success/Failure rates of individual modules.
- **User Acceptance Testing (UAT):** Feedback from actual users.
- **Performance Metrics:** Use tables to show CPU usage, memory consumption, or response times.

C. System Testing

Testing was conducted to verify that all functional requirements were met. Table I summarizes the core test cases.

TABLE I
SYSTEM TEST CASES AND RESULTS

ID	Test Scenario	Expected	Status
TC-01	User Login	Success	Passed
TC-02	Data Encryption	Ciphertext generated	Passed
TC-03	API Response	< 200ms	Passed
TC-04	DB Connection	Stable under load	Passed

IV. DISCUSSION

In this section, you answer the "So what?" question. Interpret your results. If your system was 10% faster than existing solutions, explain *why* (e.g., because of your specific database indexing strategy). Acknowledge limitations—if the system crashes with more than 500 users, state that honestly. This adds academic integrity to your project.

V. CONCLUSION

Summarize the final state of the project. Did you meet the objectives set in the Introduction? Conclude with "Future Work"—what features would you add if you had six more months? This shows that you understand the scalability of your software.

REFERENCES

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [2] A. Sharma and K. Lee, "Scalable microservices architecture for real-time data processing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1542–1555, 2023.
- [3] J. Smith and L. Brown, "Implementation of aes-256 for mobile application security," in *2024 IEEE International Conference on Cyber Security (ICCS)*. IEEE, 2024, pp. 22–29.
- [4] Meta, "React: A javascript library for building user interfaces," 2026, accessed: 2026-01-28. [Online]. Available: <https://react.dev/>